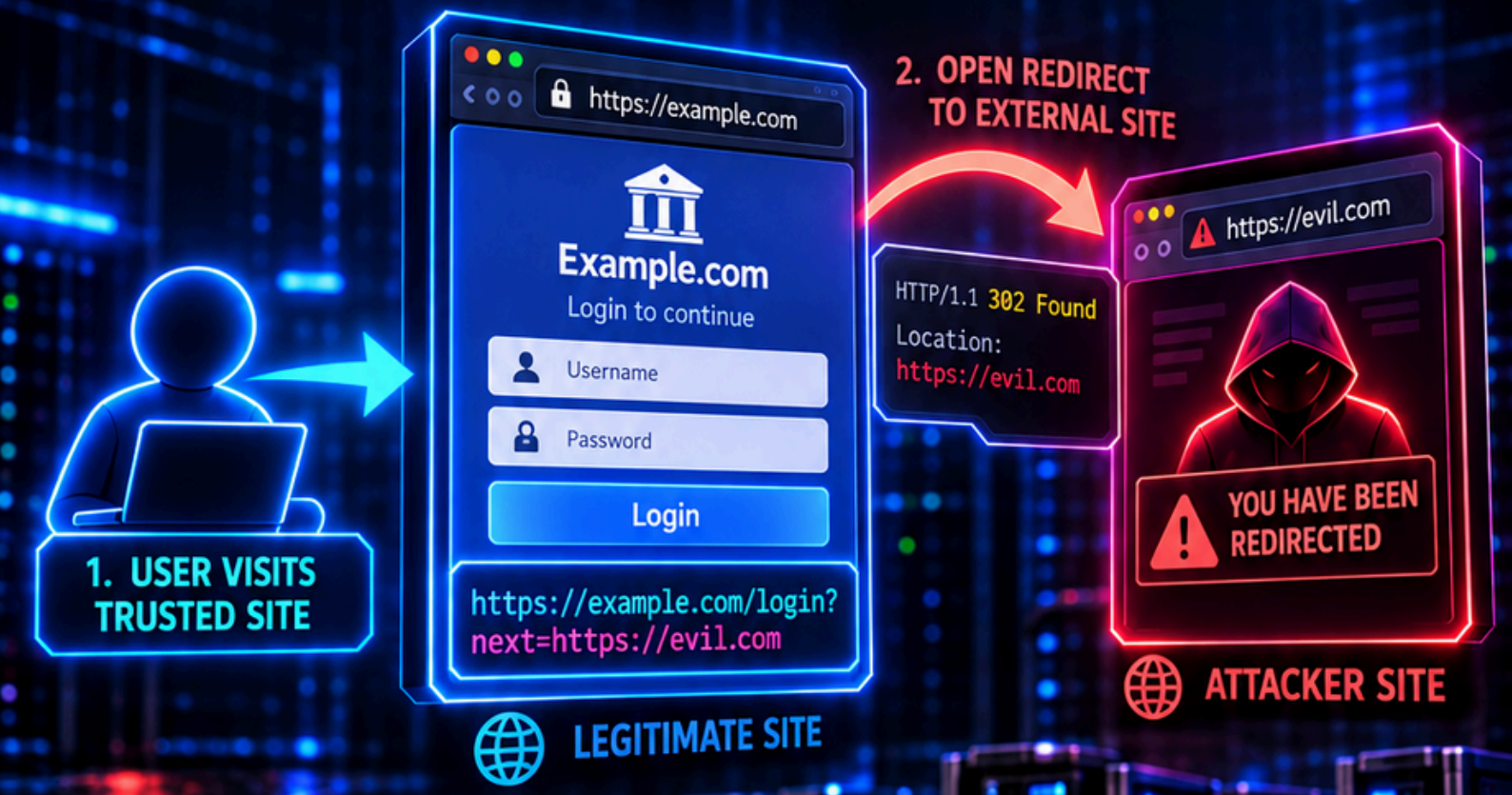
FOUNDATIONS OF WEB APP PENTESTING

11.2 OPEN REDIRECT OVERVIEW



HAROON SHAHID

FOUNDATIONS OF WEB APP PENTESTING

11.2 OPEN REDIRECT OVERVIEW

==========

OPEN REDIRECT

Definition: An open redirect is a web vulnerability that allows an attacker to make a legitimate website redirect a user to a destination the website should not allow, such as an attacker-controlled website.

A **redirect** simply means sending the browser from one URL to another.

A website redirecting users to an external website is **not automatically vulnerable**. The problem is when an attacker can choose an **unauthorized** redirect destination.

Introduction

Websites use redirects for many normal reasons, such as sending users to a login page or returning them to the page they originally wanted to visit.

Example: Returning to a page after login

1. A user visits `https://example.com/settings`.
2. The website notices that the user is not logged in.
3. It sends the user to `https://example.com/login?next=/settings`.
4. The user successfully logs in.
5. The website sends the user back to `https://example.com/settings`.

Here, `next` is a **URL parameter**. Its value, `/settings`, tells the application where the user wants to go after logging in.

Another example is `https://example.com/login?next=/dashboard`. After successful login, an application using this parameter may redirect the user to `/dashboard`.

A website **does not have to use a URL parameter** to remember the destination. It could store that information in a server-side session instead.

Where the problem begins

Suppose an attacker changes the destination to:

`https://example.com/login?next=https://attacker.example`

If the website accepts that destination without proper validation, it may send the user to `https://attacker.example` after login.

The link begins with the legitimate website, `example.com`, but the eventual destination is controlled by the attacker. That behavior may be an **open redirect**.

The link does not automatically steal the user's password. The attacker would need to trick the victim into entering sensitive information on a fake page, or exploit another weakness.

1. How Open Redirects Work

An open redirect happens when the application uses attacker-controllable input to choose a redirect destination **without correctly restricting what destinations are allowed**.

Legitimate use

`https://example.com/login?next=/dashboard`

After login, the website sends the user to an internal page: `/dashboard`.

Vulnerable use

`https://example.com/login?next=https://attacker.example`

If the site lets an attacker choose that external destination and redirects the victim there, the redirect may be vulnerable.

Two ways redirects can happen

A. Server-side (HTTP) redirect

The server returns a response such as:

HTTP/1.1 302 Found
Location: https://attacker.example

- `302 Found` is an HTTP redirect status code.
- `Location` tells the browser which URL to visit next.
- Other redirect status codes include `301`, `303`, `307`, and `308`.

B. Client-side (JavaScript) redirect

JavaScript in the webpage can also send the browser somewhere else. For example:

window.location.assign("https://attacker.example");

In this case, you might **not** see a `3xx` response with a `Location` header. You need to check what the JavaScript does in the browser.

Both types can be unsafe if an attacker can control the destination.

2. Common Parameters to Target

When hunting for open redirects, look for parameter names that suggest a destination or return path.

Common names:

- `next=`, `redirect=`, `redir=`, `url=`
- `return=`, `returnurl=`, `return_to=`
- `destination=`, `dest=`, `target=`, `continue=`
- `forward=`, `forwardurl=`, `forward_url=`
- `redirecturl=`, `redirect_url=`

Other names you may encounter:

- `uri=`, `u=`, `n=`
- `redirect_uri=`, `redirecturi=`
- `relaystate=`

These are **clues, not proof** of a vulnerability. `redirect_uri` often appears in OAuth flows, while `RelayState` is commonly used in SAML flows. A parameter's name alone does not tell you whether it causes a redirect or whether the redirect is unsafe.

Impact & Chaining

Chaining means combining one vulnerability with another weakness to produce a bigger impact.

3.1 Phishing

An attacker shares a link like:

```
https://example.com/login?next=https://attacker.example
```

A victim may trust the link because it starts with `example.com`.

If the website follows the attacker-controlled redirect, the victim reaches `attacker.example`. The attacker could display a fake login page there.

If the victim types their credentials into that **fake** page, the attacker may steal them.

Credentials entered into the **real** `example.com` login page are not automatically sent to the attacker merely because a redirect happens afterward.

The **main idea** is that an open redirect can make a phishing link look more trustworthy.

3.2 OAuth Abuse: Authorization Code or Token Theft

OAuth 2.0 is an **authorization** framework: it allows an application to receive permission to access certain resources. **OpenID Connect (OIDC)** adds authentication features and is commonly used for sign-in functions.

During an OAuth authorization flow, the authorization server sends the user's browser back to an approved address called a **redirect URI** (`redirect_uri`).

Depending on the OAuth flow, the response may contain an **authorization code** or, in some flows, an **access token**.

- **Authorization code:** A short-lived value that the client normally exchanges for tokens.
- **Access token:** A value used to access protected resources within its granted permissions.

A dangerous situation may arise if weak redirect URI validation or an open redirect on an approved callback page lets sensitive OAuth information reach an attacker-controlled destination.

However, secure OAuth servers require registered redirect URIs to be matched correctly. Other protections, such as **PKCE** and applicable client authentication, can also prevent an attacker from misusing a stolen authorization code.

An open redirect **does not automatically expose OAuth tokens or cause account takeover**. Additional weaknesses and a working exploitation path must be demonstrated.

3.3 Open Redirect → Possible XSS

XSS (Cross-Site Scripting) happens when attacker-controlled JavaScript executes in the security context of a vulnerable website.

Some webpages use JavaScript to read a redirect destination from the URL and navigate to it.

A test value might be:

```
?next=javascript:alert(document.domain)
```

If the application places this value into an unsafe JavaScript navigation operation **and the browser executes it in the legitimate website's context**, the issue may be exploitable as XSS.

Important Points:

- Merely accepting `javascript:` is **not enough** to prove XSS.
- An ordinary HTTP `Location` redirect and a JavaScript navigation operation may handle `javascript:` differently.
- JavaScript executing on `attacker.example` is **not** XSS on `example.com`.

You must confirm **where** the JavaScript executes, not only that a redirect occurs.

3.4 Open Redirect → Possible SSRF Bypass

SSRF (Server-Side Request Forgery) is a vulnerability where an attacker can influence requests made by the application's server.

Suppose a vulnerable server-side feature accepts URLs but only permits requests to `https://example.com`.

An attacker finds an open redirect on `example.com` that points to an internal system.

The possible chain is:

1. The server accepts a request to an allowed `example.com` URL.

2. That URL returns a redirect to an internal destination.
3. If the server's HTTP client follows the redirect, the server may access the internal destination.

An open redirect alone is **not SSRF**. This chain requires a server-side request feature that can be abused and follows the redirect.

Hunting Methodology

Only test websites and functions that you have permission to test. Use a domain you control, or a designated test destination, to verify the result.

Step 1 — Find Redirect Functionality

Look at actions that send users to another URL:

- Login and logout
- Return-to-page features
- Language or region selection
- Links to other pages or websites
- OAuth callbacks and other authorization flows
- Buttons such as "Continue," "Next," or "Return"

Look for destination-related parameters, but also check JavaScript and form values. Not every redirect uses a parameter named `redirect`.

Step 2 — Understand the Normal Redirect

Suppose you find:

```
https://example.com/go?next=/dashboard
```

First, observe what actually happens. Does it send you to `/dashboard`? Does the server return a redirect status? Does JavaScript change the browser's location?

For login-based redirects, **complete the login in your own test account** if the destination is only used after authentication. Do not assume the redirect happens as soon as you visit `/login`.

Step 3 — Replace the Destination

Change the value to a harmless external destination you control:

```
https://example.com/go?next=https://attacker.example
```

Here, `attacker.example` is a **fictional example domain**. For a real test, use an authorized test domain you control.

If the application sends the user to that unauthorized external destination, you may have found an open redirect.

Do not stop at seeing the parameter reflected in a response. Reflection alone does not prove redirection.

Step 4 — Confirm the Redirect in Caido or Burp

For an **HTTP redirect**:

1. Intercept the request or open it in Repeater.
2. Replace the redirect destination with your test URL.
3. Send the request. If needed, perform the action that triggers the redirect (for example, login).
4. Inspect the response status and `Location` header.

A vulnerable **example response** could be:

```
HTTP/1.1 302 Found
Location: https://attacker.example
```

This is evidence that the server instructed the browser to visit the external URL.

For a **JavaScript redirect**:

1. Open the test link in a browser.
2. Observe whether the browser changes to the unauthorized external URL.
3. Use browser Developer Tools or the JavaScript source to find what triggered the navigation.

Some clients automatically follow HTTP redirects. If that happens, inspect the **redirect chain** or turn off automatic redirect following so you can see the original response.

Step 5 — Try Validation Bypasses

If a normal external URL is blocked, investigate whether the destination validation can be bypassed.

A. Scheme-relative URL

```
?next=//attacker.example
```

`//attacker.example` is a **network-path reference**. In a normal HTTPS page context, it resolves to `https://attacker.example`.

It has no written `https:` part, but it can still point to an external host.

B. URL encoding

URL encoding represents some characters using `%` followed by hexadecimal digits.

For example:

- `:` → `%3A`
- `/` → `%2F`

Normal value:

```
?next=https://attacker.example
```

Encoded value:

`?next=https%3A%2F%2Fattacker.example`

The application may decode the encoded value before using it. If one version is blocked while another reaches the unauthorized destination, investigate whether there is a validation/decoding difference.

URL encoding does **not** automatically bypass a filter. You must verify the final redirect.

C. Fake subdomain

`?next=https://example.com.attacker.example`

A weak check might allow it because the value contains `example.com`.

But the **hostname** is `example.com.attacker.example`. The domain controlled by the attacker is `attacker.example`; this is **not** a hostname under the real `example.com` domain.

D. The `@` trick (userinfo)

`?next=https://example.com@attacker.example/`

Here, `example.com` appears **before** `@`, in the userinfo part of the URL. The **hostname is** `attacker.example`.

A filter that checks only whether the string starts with `https://example.com` might be fooled.

E. Slash and backslash handling

Some URL parsers and browsers handle slashes, backslashes, and encoded characters differently. A check that blocks only `//` may therefore be incomplete.

Do not assume a particular unusual payload works everywhere. Compare what the application accepts with the **final URL the browser actually visits**.

Testing rule: Change one thing at a time, observe the response, and verify the actual destination. A bypass is not confirmed until an unauthorized redirect works.

Step 6 — Decide Whether It Is a Real Finding

Ask yourself:

1. Can an attacker control the destination?
2. Is the resulting destination **unauthorized**, rather than an intended, approved external URL?
3. Does the application **actually redirect** there?
4. Can I reproduce it reliably and show the full request/response or browser behavior?
5. Is there a demonstrated security impact or useful exploit chain?

Bug bounty note: A standalone open redirect is often considered low severity or excluded by program policy. Do not claim account takeover, OAuth token theft, SSRF, or XSS unless you can demonstrate that impact.

Remediation / How to Fix It

5.1 Use a Fixed Redirect Destination

If the website does not need different destinations after login, the developer can choose one fixed path:

Login successful → `/dashboard`

Even if someone adds `?next=https://attacker.example`, the application ignores that input and goes to `/dashboard`.

Important: Removing redirect functionality means the parameter is **not needed**, but it does **not** mean a user cannot type the unused parameter into the URL. The application must ignore it.

Trade-off: Easy to secure, but users cannot automatically return to every page they originally requested.

5.2 Use Internal Destination IDs

Instead of accepting any URL, the server can maintain a list of approved destinations:

ID	Approved destination
1	/dashboard
2	/profile
3	/settings

Example:

1. The user tries to visit `/settings` but needs to log in.
2. The server sends the user to `/login?destination=3`.
3. After login, the server looks up ID `3`.
4. The server redirects to `/settings`.

The user does **not** need to type `3`; the application can generate it.

If an attacker submits `destination=https://attacker.example`, the server rejects it because it is not a valid ID.

Users can still change `destination=3` to `destination=2`. That is acceptable **only if** the mapped destination is approved, and normal authorization checks still protect the requested page. IDs are not a substitute for access control.

5.3 Properly Validate Internal Paths

If a website needs a parameter such as `next=/settings`, it should accept only destinations that safely resolve to **approved internal pages**.

Examples:

- `/settings` → an internal path, if permitted
- `/dashboard` → an internal path, if permitted
- `//attacker.example` → can reference an external hostname; reject
- `https://attacker.example` → external URL; reject unless specifically approved

Checking only whether the input starts with `/` is **not sufficient**. Encoded characters, `//`, backslashes, and differences between URL parsers can defeat weak checks.

Use a maintained, framework-provided **local URL validation** function, or a trusted URL parser with an appropriate allow-list. Validate the destination the browser will actually use.

5.4 Use an Allow-list for Approved Destinations

An **allow-list** is a list of redirect destinations that the application explicitly permits.

For example, an application may allow only:

- `https://example.com/dashboard`
- `https://example.com/profile`

If an application genuinely needs external redirects, it can also approve specific external destinations.

To validate an absolute URL safely, developers should compare **parsed URL components**, including the scheme, real hostname, effective port, and allowed paths where necessary.

Do **not** approve URLs merely because their text contains or starts with the trusted domain.

For example, these must not be confused with `https://example.com/`:

- `https://example.com.attacker.example/`
- `https://example.com@attacker.example/`

Avoid transforming the URL **after** validating it in a way that could change the destination.

5.5 Protect OAuth Redirects Separately

OAuth authorization servers must properly match redirect URIs against registered destinations, normally using **exact string matching** under current OAuth security guidance (with narrow specified exceptions).

OAuth applications should not expose open redirect endpoints that can leak authorization codes or tokens. Use recommended OAuth protections, including PKCE where required or appropriate.

Summary

Open Redirect: An attacker can make a legitimate website redirect a user to an **unauthorized** destination.

How it works: Attacker-controlled destination → weak validation → browser redirects somewhere it should not.

Hunting: Find redirects → understand normal behavior → replace destination → inspect `Location` or browser navigation → test bypasses → confirm the impact.

Possible impact: Phishing; OAuth abuse, XSS, or SSRF **only when additional conditions exist**.

Prevention: Fixed destinations, internal destination IDs, properly validated local paths, and allow-lists based on correctly parsed URLs.

Final reminder: An external redirect is not automatically a vulnerability, and an open redirect alone does not automatically mean account takeover.

References

1. OWASP — Unvalidated Redirects and Forwards Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html
2. PortSwigger — Open Redirection (Reflected): https://portswigger.net/kb/issues/00500100_open-redirection-reflected
3. PortSwigger — DOM-based Open Redirection: https://portswigger.net/web-security/dom-based/open-redirection
4. PortSwigger — SSRF with Filter Bypass via Open Redirection (Lab): https://portswigger.net/web-security/ssrf/lab-ssrf-filter-bypass-via-open-redirection
5. MDN — Redirections in HTTP: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Redirections
6. IETF — RFC 9700, Best Current Practice for OAuth 2.0 Security: https://www.rfc-editor.org/rfc/rfc9700.html
7. OpenID Foundation — OpenID Connect Core: https://openid.net/specs/openid-connect-core-1_0.html

==========